

CE313 Instant Messaging App — Full Codebase Analysis

App Name: SlipSpace
Course: CE313 – Computer Communication Networks (Final Project)
Stack: Python TCP server + Flutter (Dart) mobile client

Architecture Overview

[Architecture Diagram — see Markdown source]

Server-Side (Python) — `src/`

1. `protocol.py` (85 lines)

Aspect	Detail
Framing	4-byte big-endian length prefix (!I struct format)
Encoding	JSON → UTF-8
Max msg	16 MB
Exceptions	<code>ConnectionClosedError</code> , <code>PartialReadError</code>
Reliability	<code>_recvall()</code> loops until exactly <code>n</code> bytes received

2. `server.py` (136 lines)

Aspect	Detail
Binding	<code>0.0.0.0:5000</code>
Concurrency	<code>ThreadPoolExecutor(max_workers=50)</code>
Auth	5 max attempts, 30s auth timeout
Heartbeat	120s inactivity timeout (client must ping)
Shutdown	<code>signal.SIGINT</code> → <code>threading.Event</code> → graceful loop exit
Thread safety	<code>active_users_lock</code> protects the shared <code>active_users</code> dict

3. `router.py` (312 lines)

Handler	Protocol Type	Description
<code>_route_direct</code>	<code>direct_msg</code>	1-to-1 messaging with friendship check; live delivery or offline queue
<code>_route_group</code>	<code>group_msg</code>	Fan-out to all group members; per-member online/offline handling
<code>_handle_friend_action</code>	<code>friend_request</code>	Send/accept/block with user-exists validation
<code>_handle_create_group</code>	<code>create_group</code>	Creator becomes first member
<code>_handle_group_manage</code>	<code>group_manage</code>	Add/remove members (creator-only)
<code>_handle_leave_group</code>	<code>leave_group</code>	Voluntary departure
<code>_handle_get_friends</code>	<code>get_friends</code>	Returns friends + pending requests
<code>_handle_get_groups</code>	<code>get_groups</code>	Returns user's groups with creator info
<code>_handle_ack</code>	<code>ack</code>	Client confirms offline msg received → delete from DB
<code>_handle_ping</code>	<code>ping</code>	Keepalive → responds with <code>pong</code>
<code>sync_offline_messages</code>	—	On login, pushes queued messages with ACK-based deletion

4. database.py (291 lines)

Table	Key columns	Purpose
<code>users</code>	<code>username</code> (PK), <code>password_hash</code>	bcrypt-hashed credentials
<code>friendships</code>	<code>(requester, receiver)</code> , <code>status</code>	pending / accepted / blocked
<code>groups</code>	<code>group_id</code> (PK), <code>created_by</code>	Group metadata
<code>group_members</code>	<code>(group_id, username)</code>	Membership
<code>offline_messages</code>	<code>id</code> (auto), <code>sender</code> , <code>target</code> , <code>content</code> , etc.	Queued messages for offline users

[!TIP] **Security features:** Timing-safe login (dummy hash for non-existent users), foreign key enforcement, cascading deletes, context-managed transactions with auto-rollback.

Client-Side (Flutter) — `mobile_app/`

Services Layer

File	Purpose
tcp_client.dart	Raw TCP socket with length-prefix framing, mirrors <code>protocol.py</code>
connection_service.dart	High-level connection manager: heartbeat pings (55s), disconnect handling, server config persistence via <code>SharedPreferences</code>
local_storage.dart	Per-user SQLite DB (sqflite) for chat history; <code>messages</code> + <code>conversations</code> tables

State Management (Provider)

Provider	Responsibilities
AuthProvider	Login/register flow, auto-login after registration, state machine (unauthenticated → connecting → authenticating → authenticated)
ChatProvider	Message routing, receipt handling, unread counts, offline sync merging with ID dedup, local DB persistence
FriendsProvider	Friends list, pending requests, send/accept/block actions
GroupsProvider	Group CRUD, member management, leave with conversation cleanup

Screens

Screen	Key Features
AuthScreen	Tabbed login/register with gradient background, pulse animation on logo, password visibility toggle, server config shortcut
WelcomeScreen	Post-login splash with Lottie animation, staggered fade+scale, auto-nav after 2s
HomeScreen	3-tab bottom nav (Chats/Friends/Groups), badge for pending requests, swipe-to-delete chats
ChatScreen	Message list with auto-scroll, avatar grouping logic, "left group" bar, group info button
GroupDetailScreen	Group info card, add member (creator only), leave group
SettingsScreen	Server host/port config with connection tips

Widgets

Widget	Highlight
MessageBubble	WhatsApp-style left/right alignment, deterministic sender colors, shadow effects
InputBar	Frosted glass backdrop, animated send/mic button toggle with gradient, emoji & attachment placeholders
MessageStatusIcon	Custom-painted morphing animation: spinning arc → single ✓ → double ✓✓, with glow pulse for queued and shake for failed
CheckmarkPainter	<code>CustomPainter</code> with path-based checkmark drawing, arc morphing, error X
ChatTile	Conversation list item with unread badge, swipe-to-delete, smart timestamp formatting
UserAvatar	Deterministic SVG avatar via <code>multiavatar</code> package
AppLogo	Circular clipped logo with primary glow shadow

Design System

Aspect	Detail
Theme	Material 3 with <code>ColorScheme.fromSeed(0xFF6750A4)</code> (deep violet)
Typography	Google Fonts Inter (with system fallback)
Transitions	Custom <code>SmoothPageRoute</code> (slide up + fade + scale)
Dark mode	System-adaptive via <code>ThemeMode.system</code>

Test Suite — `tests/test_all.py` (663 lines)

Test Class	Coverage
<code>Test_DB_01_Users</code> (6 tests)	Register, duplicate, login pass/fail, user exists
<code>Test_DB_02_Friendships</code> (5 tests)	Send, pending check, accept, bidirectional verify, block
<code>Test_DB_03_Groups</code> (8 tests)	Create, duplicate, auto-member, creator check, add/remove member
<code>Test_DB_04_OfflineMessages</code> (4 tests)	Queue, fetch, bulk delete, single delete, group msg type
<code>Test_TCP_01_Auth</code> (5 tests)	TCP register, duplicate, login success/fail, nonexistent

<code>Test_TCP_02_PingPong</code> (2 tests)	Single + multiple pings
<code>Test_TCP_03_FriendRequests</code> (2 tests)	Nonexistent user rejection, full friend flow with notifications
<code>Test_TCP_04_DirectMessages</code> (4 tests)	Live delivery with receipt, friendship required, offline queue, sync-on-login with ACK
<code>Test_TCP_05_Groups</code> (9 tests)	Create, duplicate, add member, nonexistent user, non-admin, group msg, non-member, leave, leave non-member

[!NOTE] Integration tests spin up a real server on port 5050 with daemon threads. Each test class gets a fresh DB.

File & Size Summary

Component	Files	Total Lines
Server (<code>src/</code>)	4	~824
Tests	1	663
Flutter App (<code>mobile_app/lib/</code>)	20	~2,800
Config/Notes	5	~120
Total	30	~4,400

Strengths

- Clean separation of concerns** — Protocol, routing, and DB layers are distinct
- ACK-based offline message delivery** — Messages only deleted from queue after client confirms
- Timing-safe auth** — Prevents user enumeration via constant-time bcrypt checks
- Thread safety** — Proper lock usage around `active_users` and `pending_acks`
- Custom animations** — The `CheckmarkPainter` and `MessageStatusIcon` are well-crafted
- Per-user local DB** — Clean session isolation in the Flutter app
- Comprehensive test suite** — Both unit and integration tests with 45 test cases

Ready for your tweaks — just let me know what you'd like to change! ■